



Image, sécurité et deep learning

Compte-rendu TP3

Insertion de données cachées dans des images

Louis Jean
Master 2 IMAGINE
Université de Montpellier
N° étudiant : 21914083

23 septembre 2024

Table des matières

1	Introduction	2
1.1	Plans binaires	2
1.2	Extraction des plans binaires	2
1.3	Chiffrement d'image et comparaisons des plans binaires	3
2	Implémentation d'une méthode d'insertion	5
2.1	Génération d'une séquence aléatoire	5
2.2	Insertion du message binaire dans l'image	5
2.3	Résultats	5
3	Stéganalyse	6
4	Conclusion	7

1 Introduction

Dans ce compte-rendu, nous abordons différentes méthodes de traitement d'images dans le cadre de la stéganographie, notamment l'extraction des plans binaires et l'insertion d'un message dans les pixels d'une image en niveaux de gris.

Pour ce TP, j'ai choisi d'utiliser une photographie de vaches Aubrac prise en Lozère, que j'ai nommée *aubrac*. Cette image est au format 640x480 pixels, à l'origine au format JPG, que j'ai convertie au format PGM.



(a) aubrac.jpg



(b) aubrac.pgm

Figure 1

1.1 Plans binaires

1.2 Extraction des plans binaires

L'extraction d'un plan binaire spécifique consiste à isoler le bit correspondant de chaque pixel. Pour cela, on utilise les opérateurs binaires.

Par exemple, pour extraire le k -ième plan binaire d'un pixel, il suffit de décaler les bits du pixel de k positions vers la droite, puis de récupérer le dernier bit. Cette opération peut être réalisée avec l'opérateur de décalage droit ($\gg$) suivi d'un ET logique ($\& 1$), pour n'obtenir que la valeur du bit situé à la position k . Ce processus est répété pour tous les pixels de l'image.

```
1 for (int i = 0; i < nTaille; ++i) {  
2     ImgOut[i] = (ImgIn[i] >> k) & 1;  
3     ImgOut[i] *= 255;  
4 }
```

Le résultat est une image binaire où chaque pixel prend soit la valeur 0 (bit désactivé), soit la valeur 255 (bit activé), ce qui permet de visualiser l'information contenue dans ce plan binaire spécifique.

1.3 Chiffrement d'image et comparaisons des plans binaires

Pour cette partie, j'ai chiffré l'image *aubrac.pgm* en utilisant AES via un mode OFB, pour créer *aubrac_chiffree.pgm*.

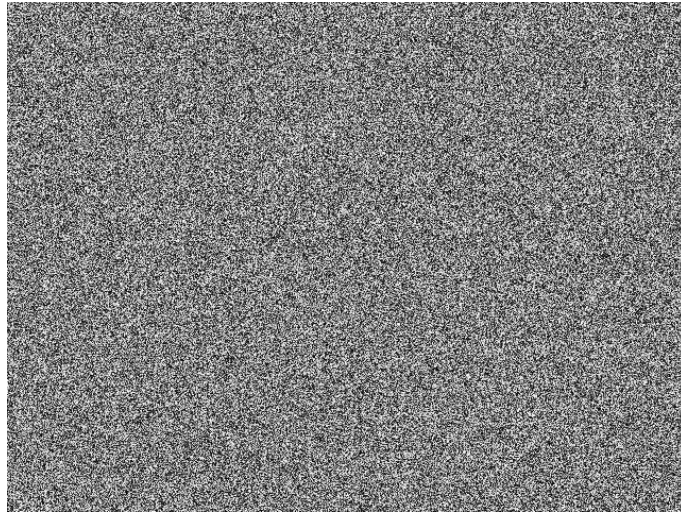


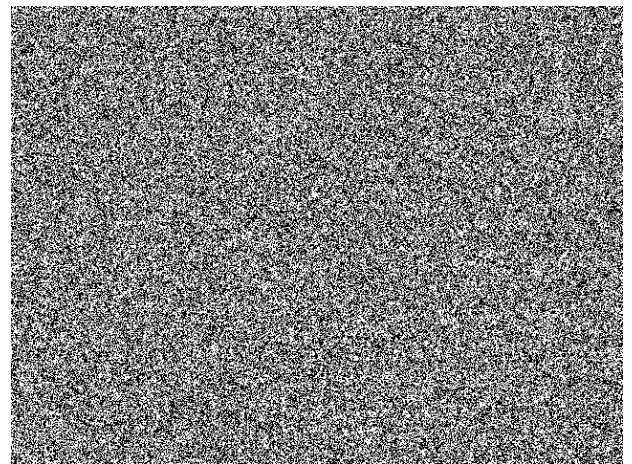
Figure 2: aubrac_chiffree.pgm

Le but était de comparer les MSB et LSB entre l'image originale et l'image chiffrée.

En ce qui concerne le plan MSB, la différence est claire : on reconnaît très bien l'image depuis l'image originale, alors qu'on ne peut établir aucun lien depuis l'image chiffrée. Ceci est cohérent car c'est le MSB qui détient le plus d'informations pour notre système visuel. De plus, il est normal de ne rien distinguer dans l'image chiffrée, car AES est conçu pour casser toute corrélation entre les pixels, rendant l'image chiffrée inexploitable.



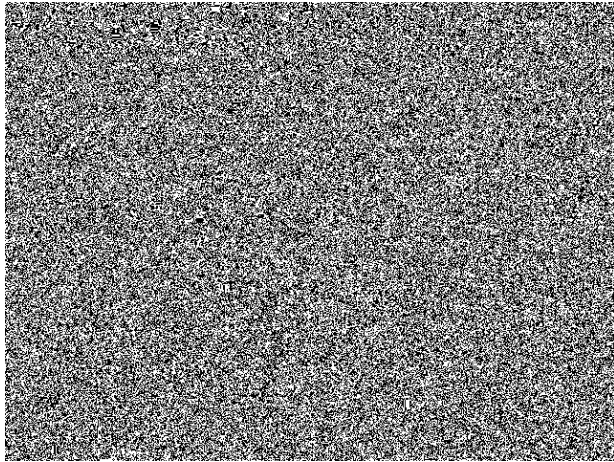
(a) aubrac_msb.pgm



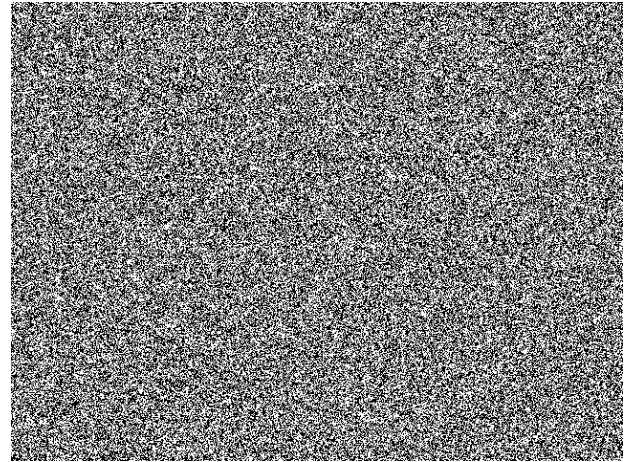
(b) aubrac_chiffree_msb.pgm

Figure 3

Au niveau du plan LSB, on ne distingue plus l'image depuis l'image originale, et toujours pas non plus depuis l'image chiffrée, puisque le plan LSB correspond aux hautes fréquences dans l'image, ce qui peut s'apparenter à du bruit. Cependant, comme la corrélation n'est pas brisée dans l'image du plan LSB de l'image originale, on peut exploiter les corrélations spatiales ou des redondances statistiques (qui ne sont pas visibles à l'œil nu) pour extraire des informations sur l'image originale.



(a) aubrac_lsb.pgm



(b) aubrac_chiffree_lsb.pgm

Figure 4

Il est important de noter que pour mon image, à partir du 3ème plan binaire (en partant du plan LSB) de l'image originale, on peut exploiter visuellement l'image, en remarquant des zones homogènes.

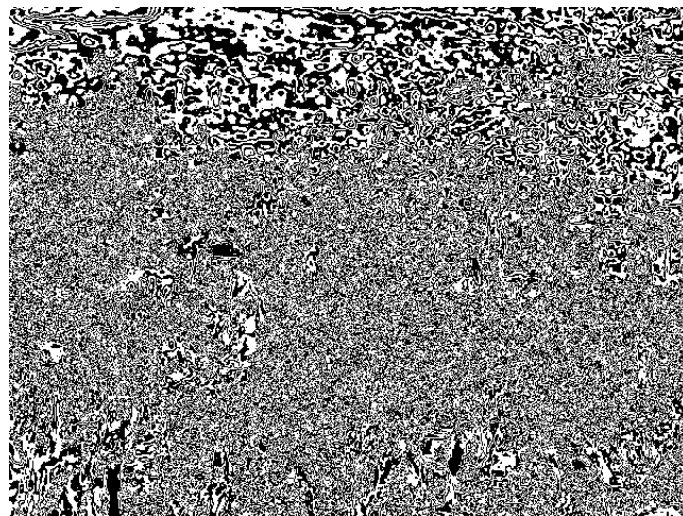


Figure 5: 3ème plan binaire de aubrac.pgm

2 Implémentation d'une méthode d'insertion

Dans cette section, j'ai implémenté une méthode permettant d'insérer une séquence binaire aléatoire dans une image en remplaçant le bit à la position k de chaque pixel.

2.1 Génération d'une séquence aléatoire

J'ai d'abord généré une séquence aléatoire de bits de taille égale à celle de l'image, en utilisant la fonction `rand()` pour produire des valeurs entières aléatoires comprises entre 0 et 1. Le code correspondant est le suivant :

```
1 srand(time(NULL));  
2 for (int i = 0; i < nTaille; ++i) {  
3     message[i] = rand() % 2;  
4 }
```

2.2 Insertion du message binaire dans l'image

L'insertion du message binaire s'effectue par substitution du bit à la position k de chaque pixel de l'image. L'opérateur binaire `&` est utilisé pour forcer le bit à 0, et l'opérateur `|` est utilisé pour insérer le bit du message dans la position souhaitée. Voici un extrait de la fonction :

```
1 for (int i = 0; i < nTaille; ++i) {  
2     ImgOut[i] = (ImgIn[i] & ~(1 << k)) | (message[i] << k);  
3 }
```

L'algorithme permet ainsi de modifier n'importe quel plan binaire de l'image, du LSB au MSB, en fonction de la position k choisie.

2.3 Résultats

Lorsque l'on injecte un message dans le plan MSB, on dégrade l'image au point de ne plus la reconnaître, ce qui n'est pas du tout ce que l'on veut dans le cadre de la stéganographie. Ici, on obtient un PSNR de 8.99142 dB entre l'image originale et l'image dont les MSB ont été substitués.

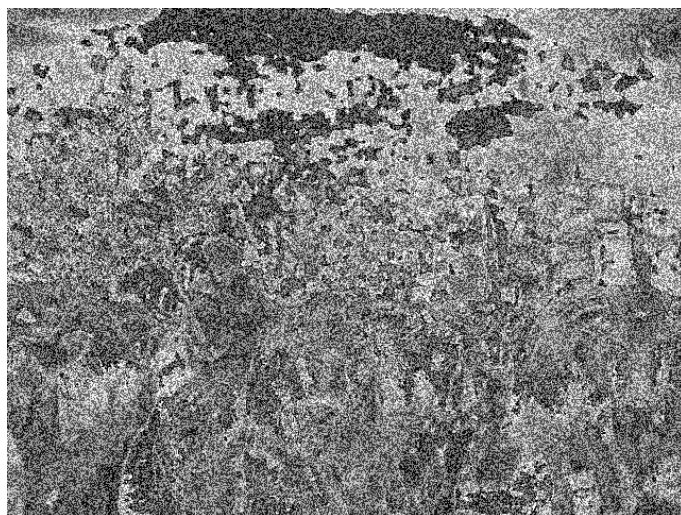


Figure 6: aubrac.pgm avec message introduit dans le plan MSB

Par contre, si l'on fait la même opération mais sur les LSB, le résultat est beaucoup plus convaincant, et la différence est totalement invisible à l'œil nu, ce qui se reflète par un PSNR de 51.1425 dB entre l'image obtenue et l'image originale.



Figure 7: aubrac.pgm avec message introduit dans le plan LSB

Ces observations font suite logique aux remarques effectuées dans la précédente partie, qui expliquaient la relation entre information contenue dans l'image et plans MSB et LSB.

3 Stéganalyse

Dans cette partie, le but était d'essayer d'analyser l'image modifiée par introduction d'un message dans le plan LSB, afin de voir si l'on peut extraire des informations de cette dernière.

À première vue, lorsque l'on trace les histogrammes, on remarque qu'ils sont assez similaires.

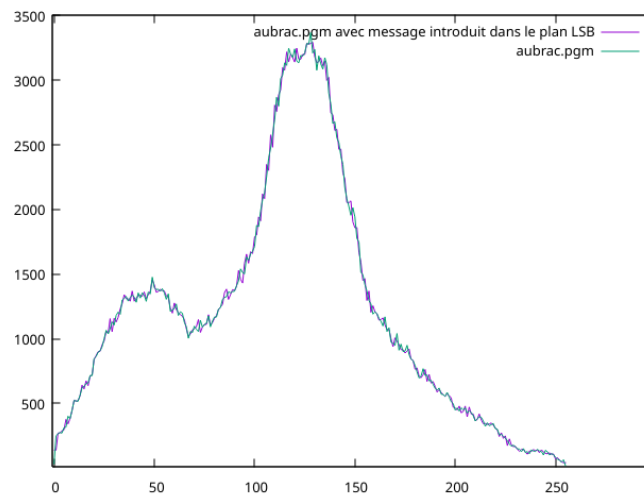


Figure 8: Histogrammes de aubrac.pgm et aubrac.insertion.lsb.pgm

Cependant, si l'on regarde de plus près, on s'aperçoit que l'introduction d'un message dans le LSB a introduit des "cassures" dans l'histogramme, en fait des plages de valeurs, ce qui est dû au fait que la distribution des LSB est rendue plus structurée par l'insertion d'un message.

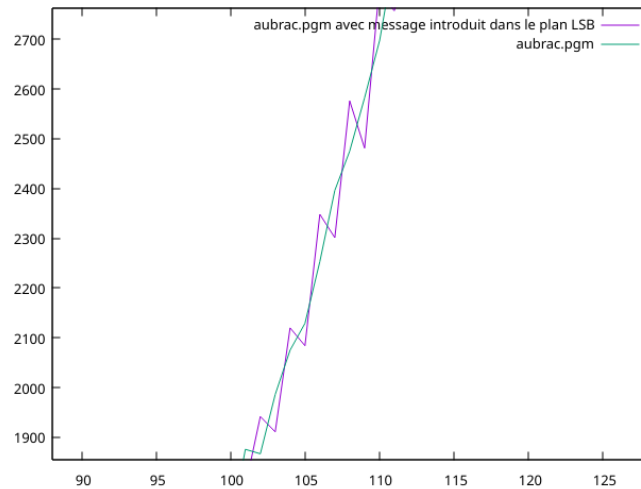


Figure 9: Histogrammes de aubrac.pgm et aubrac.insertion.lsb.pgm

La dernière question était de discuter si un attaquant pourrait détecter la présence d'un message caché dans une image marquée visuellement similaire à l'image originale. La réponse est oui, l'assaillant pourrait par exemple effectuer une analyse statistique des LSB, révélant un pattern anormal de la distribution des LSB. Par exemple, il pourrait estimer une distribution moyenne des LSB dans des images quelconques, et comparer cette distribution avec celle des LSB dans l'image suspecte.

C'est pour cette raison que, dans les échanges réels d'images marquées, il est courant de chiffrer l'image marquée après l'insertion du message, afin d'ajouter une couche supplémentaire de sécurité.

4 Conclusion

À travers ce travail, nous avons exploré la stéganographie. Nous avons d'abord extrait les plans binaires de l'image, ce qui nous a permis de constater que les informations visuelles les plus importantes sont contenues dans les bits de poids fort, tandis que les bits de poids faible peuvent être modifiés sans grand impact visuel. Ce TP a montré l'importance de comprendre non seulement les techniques de dissimulation d'information, mais aussi les méthodes pour les détecter.